```
#!/usr/bin/env python3
"""
VA BDD Claim Preparation Tool
==============================
Helps veterans prepare Benefits Delivery at Discharge (BDD) claims
with maximum rating potential under 38 CFR Part 4.

Usage:
    python main.py

Requirements:
    pip install rich cryptography anthropic reportlab questionary python-dateutil

Author: VA BDD Tool
License: For veteran use — not legal or medical advice.
"""

import sys
import os
import json
from pathlib import Path
from typing import Dict, List, Optional

from rich.console import Console
from rich.panel import Panel
from rich.text import Text
from rich.table import Table
from rich.progress import Progress, SpinnerColumn, TextColumn
from rich import box
import questionary
from questionary import Style

# ── Project Imports ──────────────────────────────────────────
sys.path.insert(0, str(Path(__file__).parent))

from modules.secure_storage import SecureStorage, prompt_password_setup
from modules.eligibility import run_eligibility_check
from modules.condition_intake import (
    select_conditions_to_claim,
    collect_condition_symptoms,
    run_secondary_condition_checks,
)
from modules.rating_engine import compute_full_rating, estimate_monthly_compensat
from modules.ai_language import (
```

```
    generate_condition_description,
    generate_nexus_letter_framework,
    generate_cp_exam_prep,
    interactive_language_review,
)
from modules.outputs import choose_output_directory, generate_all_outputs
from data.conditions_db import CONDITIONS_DB

# ═══════════════════════════════════════════════════════════════════════
console = Console()

STYLE = Style([
    ("qmark", "fg:cyan bold"),
    ("question", "bold"),
    ("answer", "fg:green bold"),
    ("pointer", "fg:cyan bold"),
    ("highlighted", "fg:cyan bold"),
    ("selected", "fg:green"),
    ("separator", "fg:yellow"),
])

APP_VERSION = "1.0.0"
APP_TITLE = "VA BDD Claim Preparation Tool"

MAIN_MENU_CHOICES = [
    "📋  Start New Claim",
    "📂  Resume Saved Claim",
    "📊  View Rating Calculator",
    "📁  Generate Output Documents",
    "⚙️   Settings",
    "❌  Exit",
]

# ═══════════════════════════════════════════════════════════════════════
# Splash Screen
# ═══════════════════════════════════════════════════════════════════════

def print_splash():
    console.clear()
    console.print(Panel(
        Text.from_markup(
            f"\n"
            f"[bold cyan]  ██╗   ██╗ ██████╗     ██████╗ ██████╗ ██████╗ \n"
            f"  ██║   ██║██╔═══██╗   ██╔═══██╗██╔═══██╗██╔═══██╗\n"
            f"  ██║   ██║███████║   ██████╔╝██║  ██║██║  ██║\n"
            f"  ╚██╗ ██╔╝██╔═══██║   ██╔═══██╗██║  ██║██║  ██║\n"
            f"   ╚████╔╝ ██║   ██║   ██████╔╝██████╔╝██████╔╝\n"
```

```
            f"    ╚═══╝  ╚═╝  ╚═╝   ╚══════╝ ╚══════╝ ╚══════╝ \n"
            f"\n"
            f"[white]       Benefits Delivery at Discharge — Claim Preparation To
            f"[dim]                              Version {APP_VERSION}[/dim]\n"
        ),
        border_style="cyan",
        padding=(0, 2),
    ))

    console.print(Panel(
        "[bold yellow]⚖  LEGAL NOTICE[/bold yellow]\n\n"
        "This tool helps you prepare and organize your VA disability claim.\n"
        "It is [bold]NOT[/bold] a substitute for accredited legal, medical, or cl
        "• All generated language is based on YOUR reported symptoms.\n"
        "  Never submit information that is inaccurate or exaggerated.\n\n"
        "• Consult a [bold]VSO (Veterans Service Organization)[/bold] for profess
        "  Free VSO services: DAV | VFW | American Legion | AMVETS | WWP\n\n"
        "• Your data is encrypted locally. Nothing is sent to any server\n"
        "  except the AI language generation calls to the Claude API.",
        border_style="yellow",
        padding=(0, 1),
    ))


# ═══════════════════════════════════════════════════════════════════════
# Main Workflow
# ═══════════════════════════════════════════════════════════════════════

def run_new_claim(storage: SecureStorage) -> Dict:
    """
    Execute the full claim preparation workflow:
    1. BDD eligibility check
    2. Veteran profile
    3. Condition selection and symptom intake
    4. Secondary condition check
    5. Rating calculation
    6. AI language generation
    7. Output generation
    """
    claim_data = storage.get("current_claim", {}) or {}

    # ── STEP 1: Eligibility ──────────────────────────────────────────
    console.print()
    run_step = questionary.confirm(
        "Step 1: Run BDD Eligibility Check?",
        default=True,
        style=STYLE,
```

```
).ask()

if run_step:
    eligibility = run_eligibility_check(storage)
    claim_data["eligibility"] = eligibility
    claim_data["service_info"] = storage.get("service_info", {})
    _save_progress(storage, claim_data)

# ── STEP 2: Condition Intake ──────────────────────────────────────
console.print()
console.print(Panel(
    "[bold cyan]Step 2: Select and Document Your Conditions[/bold cyan]\n\n"
    "You'll select conditions from the 38 CFR Part 4 schedule and\n"
    "document your symptoms for each one.",
    border_style="cyan"
))

selected_conditions = select_conditions_to_claim()

# ── STEP 3: Secondary Conditions ──────────────────────────────────
if selected_conditions:
    secondary = run_secondary_condition_checks(selected_conditions)
    all_conditions = selected_conditions + secondary
else:
    all_conditions = []

if not all_conditions:
    console.print("[yellow]No conditions selected. Returning to main menu.[/y
    return claim_data

# ── STEP 4: Symptom Interview (per condition) ─────────────────────
console.print()
console.print(Panel(
    f"[bold cyan]Step 3: Symptom Documentation[/bold cyan]\n\n"
    f"You'll be guided through a detailed symptom interview for each of your\
    f"[bold]{len(all_conditions)}[/bold] condition(s).\n\n"
    "[dim]This is the most important section — thorough answers here directly
    "determine your rating tier and the quality of your claim language.[/dim]
    border_style="cyan"
))

service_info = claim_data.get("service_info", {})
documented_conditions = []
bilateral_pairs_raw = []

for i, cond_key in enumerate(all_conditions, 1):
    cond_name = CONDITIONS_DB.get(cond_key, {}).get("name", cond_key)
```

```
    console.print()
    console.rule(f"[cyan]Condition {i} of {len(all_conditions)}: {cond_name}[

    result = collect_condition_symptoms(cond_key, service_info)
    if result:
        documented_conditions.append(result)

        # Check for bilateral pairs already documented
        pair = result.get("bilateral_pair")
        if pair:
            # Check if the paired condition was also documented
            for existing in documented_conditions:
                if existing["condition_key"] == pair:
                    # Register the bilateral pair
                    pair_already_registered = any(
                        (a == cond_key and b == pair) or (a == pair and b ==
                        for a, b, _, _ in bilateral_pairs_raw
                    )
                    if not pair_already_registered:
                        bilateral_pairs_raw.append((
                            cond_key,
                            pair,
                            result["estimated_rating"],
                            existing["estimated_rating"],
                        ))

    _save_progress(storage, {**claim_data, "conditions": documented_condition

claim_data["conditions"] = documented_conditions

# ── STEP 5: Combined Rating Calculation ──────────────────────────
console.print()
console.print(Panel(
    "[bold cyan]Step 4: Combined Rating Calculation[/bold cyan]\n\n"
    "Calculating your estimated combined VA disability rating using the\n"
    "38 CFR § 4.25 whole-person method, including bilateral factor (§ 4.26)."
    border_style="cyan"
))

bilateral_pairs = []
standalone = []
bilateral_cond_keys = set()

for a_key, b_key, a_rating, b_rating in bilateral_pairs_raw:
    a_name = CONDITIONS_DB.get(a_key, {}).get("name", a_key)
    b_name = CONDITIONS_DB.get(b_key, {}).get("name", b_key)
    bilateral_pairs.append((a_name, b_name, a_rating, b_rating))
```

```
    bilateral_cond_keys.add(a_key)
    bilateral_cond_keys.add(b_key)

for cond in documented_conditions:
    if cond["condition_key"] not in bilateral_cond_keys:
        standalone.append(cond["estimated_rating"])

rating_result = compute_full_rating(standalone, bilateral_pairs)
claim_data["rating_result"] = rating_result

comp_estimate = estimate_monthly_compensation(rating_result["combined_final"]
claim_data["compensation_estimate"] = comp_estimate

_display_rating_summary(rating_result, comp_estimate, documented_conditions)

# ── STEP 6: AI Language Generation ──────────────────────────────
console.print()
use_ai = questionary.confirm(
    "Step 5: Generate AI-optimized claim language for each condition?",
    default=True,
    style=STYLE,
).ask()

ai_language = {}
nexus_letters = {}
cp_prep = None

if use_ai:
    api_key = os.environ.get("ANTHROPIC_API_KEY")
    if not api_key:
        api_key = questionary.password(
            "Enter your Anthropic API key (for AI language generation):",
            style=STYLE,
        ).ask()
        if api_key:
            os.environ["ANTHROPIC_API_KEY"] = api_key

    if api_key:
        for cond in documented_conditions:
            cond_key = cond["condition_key"]
            cond_db = CONDITIONS_DB.get(cond_key, {})

            console.print(f"\n[cyan]Generating language for: {cond['condition

            while True:
                generated = generate_condition_description(
                    condition_name=cond["condition_name"],
```

```
                diagnostic_code=cond["diagnostic_code"],
                cfr_reference=cond.get("cfr_reference", "38 CFR Part 4"),
                symptoms=cond["symptoms"],
                rating_criteria=cond_db.get("rating_criteria", {}),
                service_info=service_info,
                target_rating=cond["estimated_rating"],
            )

            display_text = generated.get("form_language", "")
            result = interactive_language_review(
                label=cond["condition_name"],
                generated_text=display_text,
                field_name="526EZ Claim Language",
            )

            if result == "REGENERATE":
                continue
            elif result:
                generated["form_language"] = result

            ai_language[cond_key] = generated
            break

    # Nexus letter
    gen_nexus = questionary.confirm(
        f"Generate a nexus letter framework for {cond['condition_name
        default=True,
        style=STYLE,
    ).ask()

    if gen_nexus:
        nexus_text = generate_nexus_letter_framework(
            condition_name=cond["condition_name"],
            diagnostic_code=cond["diagnostic_code"],
            service_info=service_info,
            in_service_events=cond.get("in_service_events", []),
        )
        nexus_result = interactive_language_review(
            label=f"{cond['condition_name']} — Nexus Letter",
            generated_text=nexus_text,
            field_name="Nexus Letter Framework",
        )
        if nexus_result and nexus_result != "REGENERATE":
            nexus_letters[cond_key] = nexus_result

# C&P Exam Prep
console.print("\n[cyan]Generating C&P exam preparation guide...[/cyan
```

```
            cp_prep = generate_cp_exam_prep(documented_conditions, service_info)
            console.print("[green]✓ C&P prep guide ready.[/green]")

        else:
            console.print("[yellow]Skipping AI generation — no API key provided.[

    claim_data["ai_language"] = ai_language
    claim_data["nexus_letters"] = nexus_letters
    _save_progress(storage, claim_data)

    # ── STEP 7: Output Generation ──────────────────────────────
    console.print()
    gen_outputs = questionary.confirm(
        "Step 6: Generate your claim documents (PDF, 526EZ language, C&P prep, ne
        default=True,
        style=STYLE,
    ).ask()

    if gen_outputs:
        output_dir = choose_output_directory()
        console.print(f"\n[cyan]Generating documents in:[/cyan] {output_dir}\n")

        files = generate_all_outputs(
            claim_data=claim_data,
            output_dir=output_dir,
            ai_language=ai_language if ai_language else None,
            cp_exam_prep=cp_prep,
        )

        console.print()
        console.print(Panel(
            "[bold green]✅ Claim Package Complete[/bold green]\n\n"
            f"Generated [bold]{len(files)}[/bold] document(s) in:\n"
            f"[cyan]{output_dir}[/cyan]\n\n"
            "NEXT STEPS:\n"
            "  1. Review all generated documents carefully.\n"
            "  2. Take the 526EZ language to va.gov or your VSO to file.\n"
            "  3. Schedule your C&P exams — reference the prep sheet.\n"
            "  4. Give nexus letter frameworks to your treating physicians.\n"
            "  5. Store these files securely — they contain your PHI.",
            border_style="green",
        ))

    return claim_data


# ═══════════════════════════════════════════════════════════════════
```

```
# Rating Calculator (Standalone)
# ──────────────────────────────────────────────────────────────────────

def run_rating_calculator():
    """Interactive standalone combined ratings calculator."""
    console.print()
    console.print(Panel(
        "[bold cyan]VA Combined Ratings Calculator[/bold cyan]\n"
        "38 CFR § 4.25 — Whole Person Method\n\n"
        "Enter your individual condition ratings below.\n"
        "Type 'done' when finished.",
        border_style="cyan"
    ))

    ratings = []
    bilateral_pairs = []

    while True:
        console.print(f"\n  Current ratings: {ratings}")
        entry = questionary.text(
            "Add a rating (0-100) or type 'bilateral' / 'done':",
            style=STYLE,
        ).ask()

        if not entry or entry.lower() == "done":
            break
        elif entry.lower() == "bilateral":
            try:
                left = int(questionary.text("Left limb rating:", style=STYLE).ask
                right = int(questionary.text("Right limb rating:", style=STYLE).a
                bilateral_pairs.append(("Left", "Right", left, right))
                console.print(f"[green]Added bilateral pair: {left}% / {right}%[/
            except (ValueError, TypeError):
                console.print("[red]Invalid values.[/red]")
        else:
            try:
                val = int(entry.strip().replace("%", ""))
                if 0 <= val <= 100:
                    ratings.append(val)
                    console.print(f"[green]Added: {val}%[/green]")
                else:
                    console.print("[red]Enter a value between 0 and 100.[/red]")
            except ValueError:
                console.print("[red]Invalid input — enter a number.[/red]")

    if not ratings and not bilateral_pairs:
        console.print("[yellow]No ratings entered.[/yellow]")
```

```
        return

    result = compute_full_rating(ratings, bilateral_pairs)
    comp = estimate_monthly_compensation(result["combined_final"])
    _display_rating_summary(result, comp, [])


# ──────────────────────────────────────────────────────────────────────
# Display Helpers
# ──────────────────────────────────────────────────────────────────────

def _display_rating_summary(rating_result: Dict, comp: Dict, conditions: List):
    """Display a rich formatted rating summary panel."""
    table = Table(box=box.ROUNDED, show_header=True, header_style="bold cyan", pa
    table.add_column("Item", style="bold")
    table.add_column("Value", style="white", justify="right")

    if conditions:
        for c in conditions:
            table.add_row(c["condition_name"], f"{c['estimated_rating']}%")
        table.add_row("", "")

    if rating_result.get("bilateral_adjustments"):
        for adj in rating_result["bilateral_adjustments"]:
            table.add_row(
                f"Bilateral: {adj['left'][0]} + {adj['right'][0]}",
                f"{adj['left'][1]}% + {adj['right'][1]}% + bilateral bonus → {adj
            )
        table.add_row("", "")

    combined = rating_result.get("combined_final", 0)
    color = "bold green" if combined >= 70 else "green" if combined >= 50 else "y
    table.add_row("Raw Combined", f"{rating_result.get('combined_raw', 0):.2f}%")
    table.add_row(f"[{color}]COMBINED RATING[/{color}]", f"[{color}]{combined}%[/
    table.add_row("Est. Monthly (no dep.)", f"${comp['base_monthly']:,.2f}")
    table.add_row("Est. Annual", f"${comp['annual_estimate']:,.2f}")

    console.print()
    console.print(table)

    if rating_result.get("tdiu_eligible"):
        console.print(Panel(
            f"[bold green]💡 TDIU ELIGIBLE[/bold green]\n{rating_result['tdiu_rea
            "TDIU pays at the 100% rate. File VA Form 21-8940 if unable to work."
            border_style="green",
        ))
```

```
def _save_progress(storage: SecureStorage, claim_data: Dict):
    """Autosave claim progress to encrypted storage."""
    storage.set("current_claim", claim_data)


# ──────────────────────────────────────────────────────────────────────
# Main Entry Point
# ──────────────────────────────────────────────────────────────────────

def main():
    print_splash()

    # ── Secure Storage Setup ──────────────────────────────────────────
    storage_dir = questionary.text(
        "Where do you want to store your encrypted claim data?",
        default=str(Path.home() / ".va_bdd_tool"),
        instruction="(press Enter to use the default — stored only on this comput
        style=STYLE,
    ).ask()

    if not storage_dir:
        storage_dir = str(Path.home() / ".va_bdd_tool")

    storage = SecureStorage(Path(storage_dir))
    success = prompt_password_setup(storage)

    if not success:
        console.print("[red]Could not open secure storage. Exiting.[/red]")
        sys.exit(1)

    console.print(f"[dim]Encrypted data stored at: {storage.storage_location()}[/

    # ── Main Menu Loop ────────────────────────────────────────────────
    while True:
        console.print()
        console.rule("[cyan]Main Menu[/cyan]")

        choice = questionary.select(
            "What would you like to do?",
            choices=MAIN_MENU_CHOICES,
            style=STYLE,
        ).ask()

        if choice is None or "Exit" in choice:
            console.print("\n[cyan]Your claim data is saved and encrypted. Goodby
            sys.exit(0)
```

```
        elif "Start New Claim" in choice:
            run_new_claim(storage)

        elif "Resume Saved Claim" in choice:
            saved = storage.get("current_claim")
            if saved:
                console.print("[green]✓ Loaded your saved claim.[/green]")
                action = questionary.select(
                    "Continue from where you left off?",
                    choices=["Continue claim workflow", "View saved data", "Start
                    style=STYLE,
                ).ask()
                if action and "Continue" in action:
                    run_new_claim(storage)
                elif action and "View" in action:
                    console.print_json(json.dumps(saved, indent=2, default=str))
                elif action and "Start fresh" in action:
                    confirm = questionary.confirm("This will erase saved progress
                    if confirm:
                        storage.delete("current_claim")
                        storage.delete("eligibility")
                        storage.delete("service_info")
                        console.print("[yellow]Saved data cleared.[/yellow]")
            else:
                console.print("[yellow]No saved claim found. Starting a new one.[
                run_new_claim(storage)

        elif "Rating Calculator" in choice:
            run_rating_calculator()

        elif "Generate Output" in choice:
            saved = storage.get("current_claim")
            if saved:
                output_dir = choose_output_directory()
                ai_lang = saved.get("ai_language")
                cp_prep = None
                generate_all_outputs(saved, output_dir, ai_lang, cp_prep)
            else:
                console.print("[yellow]No saved claim data to export. Run a claim

        elif "Settings" in choice:
            settings_menu(storage)


def settings_menu(storage: SecureStorage):
    """Settings and data management."""
```

```
    choice = questionary.select(
        "Settings",
        choices=[
            "📁  Change output directory preference",
            "🗑️  Clear all saved claim data",
            "📤  Export claim data to plain text (for VSO use)",
            "↩   Back to main menu",
        ],
        style=STYLE,
    ).ask()

    if choice and "Export" in choice:
        export_path = questionary.text(
            "Enter export file path (e.g., /Users/you/Desktop/my_claim.json):",
            style=STYLE,
        ).ask()
        if export_path:
            success = storage.export_plaintext(Path(export_path))
            if success:
                console.print(f"[green]✓ Exported to {export_path}[/green]")
                console.print("[yellow]⚠  This file is UNENCRYPTED — handle it se
            else:
                console.print("[red]Export failed. Check the path and try again.[

    elif choice and "Clear" in choice:
        confirm = questionary.confirm(
            "This will permanently delete ALL saved claim data. Are you sure?",
            default=False,
            style=STYLE,
        ).ask()
        if confirm:
            storage.wipe_all()
            console.print("[yellow]All data cleared.[/yellow]")


if __name__ == "__main__":
    main()
```